Recipiente com Mais Água

Pesquisa e Análise – Estrutura de Dados

João Pedro de Nobile

João Victor Francelino

Por que esse problema existe?

- Em muitas situações, queremos maximizar uma área ou quantidade com base em limites, como paredes, recipientes ou distâncias.
- O problema ‘Recipiente com Mais Água’ simula isso: encontrar duas linhas que formem o maior “recipiente” possível.
- É um desafio clássico de otimização e raciocínio lógico, usado em entrevistas de empresas.

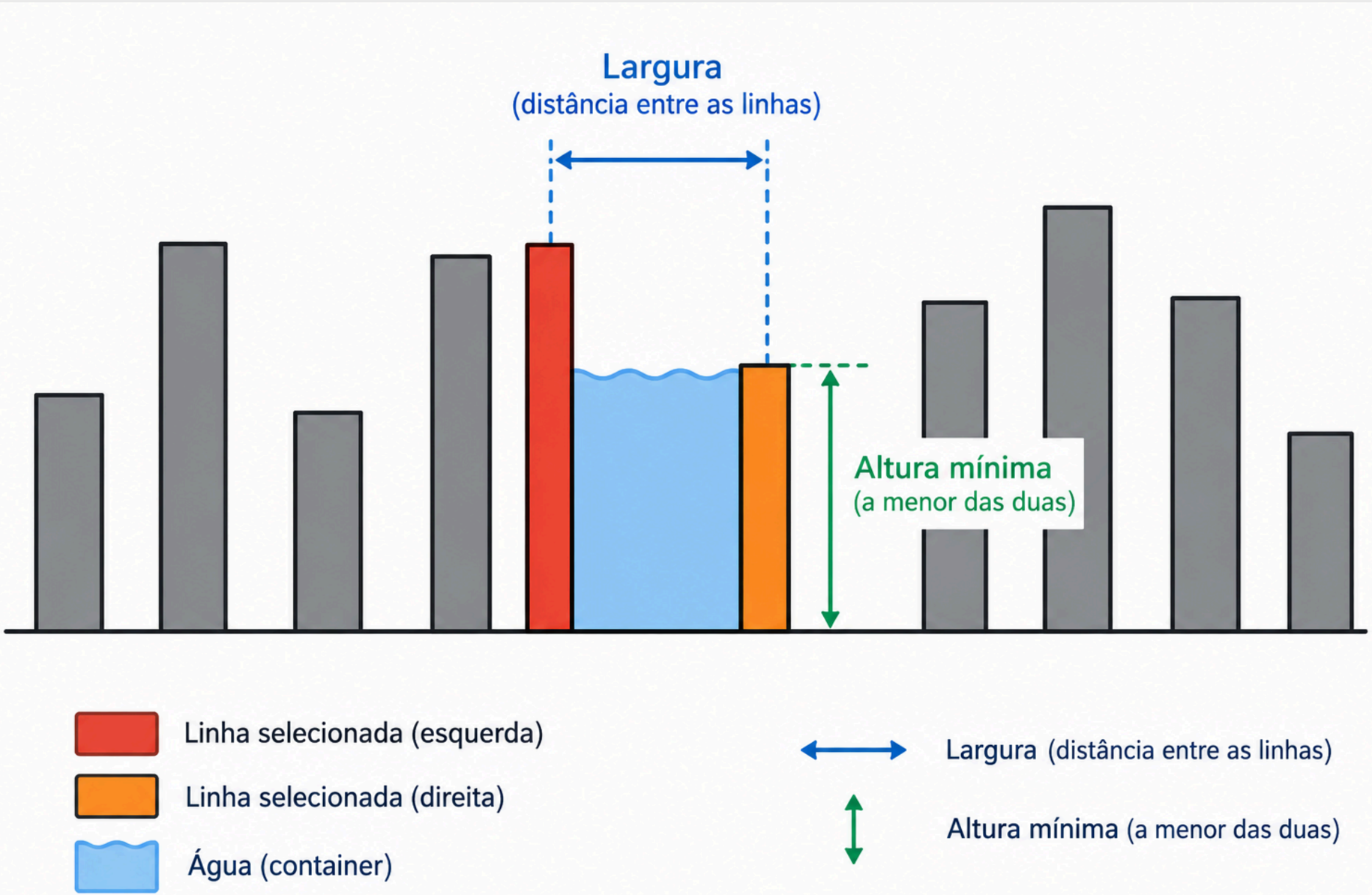
Entendendo o problema

- Dado um array de alturas, cada elemento representa uma linha vertical.
- Precisamos escolher duas linhas que formem o maior container possível.
- A área é calculada por:

Área = largura × altura mínima

largura = distância entre os índices escolhidos

altura mínima = menor altura entre as duas linhas


Largura
(distância entre as linhas)
Altura mínima
(a menor das duas)
Linha selecionada (esquerda)
Linha selecionada (direita)
Água (container)
Largura (distância entre as linhas)
Altura mínima (a menor das duas)

Força Bruta

A abordagem brute force resolve o problema testando todas as combinações possíveis de pares de linhas. Para cada índice i, percorremos todos os índices j > i e calculamos:

$$\text{área} = \min(\text{height}[i], \text{height}[j]) \times (j - i)$$

Em um array com 9 elementos, isso significa verificar 36 pares diferentes. A implementação é feita com dois loops aninhados, simples e direta, o que facilita provar a correção.

Solução Ingênua (Brute Force)

```c
int maxArea(int* height, int heightSize) {
    int max = 0;
    for (int i = 0; i < heightSize - 1; i++) {
        for (int j = i + 1; j < heightSize; j++) {
            int h = (height[i] < height[j]) ? height[i] : height[j];
            int area = h * (j - i);
            if (area > max) {
                max = area;
            }
        }
    }
    return max;
}
```

Tempo

$O(n^2)$

Espaço

$O(1)$

Observação: resolve o problema, mas é impraticável para grandes entradas.

Dois Ponteiros

A técnica de dois ponteiros usa um ponteiro no início e outro no fim do array para explorar as possíveis áreas. A área formada entre duas linhas é limitada pela menor altura entre elas.

Se movermos o ponteiro da maior altura, a altura mínima não aumenta e ainda reduzimos a largura, logo, não há ganho. Por isso, sempre deslocamos o ponteiro que aponta para a altura menor, preservando a chance de encontrar uma área maior.

Solução Otimizada (Two Pointers)

```c
int maxArea(int* height, int heightSize) {
    int left = 0;
    int right = heightSize - 1;
    int max = 0;

    while (left < right) {
        int h = (height[left] < height[right]) ? height[left] : height[right];
        int area = h * (right - left);
        if (area > max) {
            max = area;
        }

        // Move o ponteiro menor
        if (height[left] < height[right]) {
            left++;
        } else {
            right--;
        }
    }

    return max;
}
```

Tempo

O(n)

Espaço

O(1)

Observação: solução esperada em entrevistas de Big Tech.

Two Pointers: Aplicação

Implementar solução

Codifique a solução Two Pointers e valide com casos extremos (entrada vazia, números grandes, overflow evitado).

Casos extremos

Teste ordens crescente e decrescente, além de valores repetidos, para garantir robustez do algoritmo.

Resolver variações

Pratique variações do problema: contar pares com soma alvo em array ordenado; explorar restrições adicionais para maximizar resultados.

Comparativo

Abordagem	Tempo	Espaço	Observação
Brute Force	$O(n^2)$	O(1)	Testa todas as combinações possíveis
Two Pointers	O(n)	O(1)	Estratégia eficiente e prática, usada em entrevistas

```
Quantidade de elementos: 100000

--- FORCA BRUTA O(n2) ---
Maior area encontrada: 89976,60
Tempo de execucao: 31,662000 segundos
Quantidade de repeticoes: 4999950000

--- DOIS PONTEIROS O(n) ---
Maior area encontrada: 89976,60
Tempo de execucao: 0,000000 segundos
Quantidade de repeticoes: 99999

================================================
                COMPARACAO
================================================

Diferenca de tempo: 31,662000 segundos
Diferenca de repeticoes: 4999850001

Os dois algoritmos encontraram o mesmo resultado.
```



Conclusão

- Ambas as soluções resolvem o problema corretamente.
- Diferença está na eficiência: brute force é didático, mas impraticável.
- Two Pointers é a solução que Big Tech espera em entrevistas.

Referências

Problema original (LeetCode #11 – Container With Most Water)
LeetCode – Container With Most Water (leetcode.com in Bing)
Editorial detalhado com análise de brute force e two pointers
The DSA Handbook – Container With Most Water editorial
Guia de entrevistas mostrando empresas que usam esse problema (Google, Amazon, Meta, Microsoft, etc.)
DSA Prep – Container With Most Water: Optimal O(n) Solution
Explicação geral da técnica Two Pointers
GeeksforGeeks – Two Pointers Technique
Generalist Programmer – Two Pointers Technique (with Examples)
AlgoArk Blog – Two Pointers Technique Explained
Comparação entre brute force e soluções otimizadas em entrevistas
ByteByteGo – Optimized vs. Brute Force Approaches